

Knowledge Distillation Across Teacher Architectures for Edge-Efficient Plant Disease Classification

Abdulkali Çeltik

Department of Computer Engineering
İzmir Institute of Technology
İzmir, Türkiye
abdulkaliceltik@std.iyte.edu.tr

Meriç Kelemeden

Department of Computer Engineering
İzmir Institute of Technology
İzmir, Türkiye
merickelemeden@std.iyte.edu.tr

Abstract—Accurate image-based plant disease classification supports precision agriculture, but high-capacity networks are inconvenient for edge or mobile deployment. This work studies knowledge distillation from large teachers into a compact MobileNetV3-Small student, asking in particular whether the *architecture of the teacher* affects the distilled student. As a preliminary demonstration on the 38-class PlantVillage dataset, a ResNet50 teacher distills into a student reaching $99.48 \pm 0.05\%$ accuracy with 1.56M parameters, about a fifteenth as many as the teacher, and with a twelvefold smaller seed-to-seed standard deviation than the same architecture trained supervised — so distillation stabilizes the compact student as well as improving it. The main study uses the smaller, class-imbalanced Cassava leaf-disease dataset (five classes) under stratified five-fold cross-validation: three teachers from different architectural families — ResNet50 (residual CNN), EfficientNet-B2 (compound-scaled CNN), and ViT-B/16 (vision transformer) — are each retrained per fold and distilled into the same student. Distillation improves the student over a supervised baseline by 1.4–1.8 accuracy points, but the choice of teacher has little effect: the three students agree to within 0.5 points despite the teachers differing widely in accuracy (82.4–85.1%) and size (7.7M–85.8M parameters). The benefit of distillation thus appears to come from the procedure itself rather than from the capacity or architecture of the teacher. Naive post-training int8 quantization of the student, however, collapses it: five distilled students fall to $29.9 \pm 13.2\%$, far below the 62% majority-class rate, while the same pipeline quantizes the ResNet50 teacher losslessly. Quantizing one block at a time localizes the failure precisely — to just two blocks, the input convolution and the first bottleneck, which cost 57 and 39 accuracy points alone while the other twelve cost at most 0.2 each. Keeping those two blocks (0.08% of parameters) in float32 makes post-training quantization essentially lossless (83.9%, within about 0.4 points of float32) with no retraining, and outperforms quantization-aware training ($80.7 \pm 0.4\%$ after 30 fine-tuning epochs) at a fraction of the cost. The same two-block exemption transfers unchanged to the 38-class PlantVillage student, recovering it from a 37.7% collapse to 99.4%, which indicates the effect is a property of the architecture rather than of one dataset. The broad lesson is that an int8 MobileNetV3-class model is reachable, but only if two identifiable blocks are excluded, and that its raw post-training accuracy is so sensitive to the calibration draw that single-number int8 results for this

class of architecture are unsound.

Index Terms—Knowledge distillation, plant disease classification, MobileNetV3, post-training quantization, cross-validation, vision transformer, edge inference

I. INTRODUCTION

Plant disease recognition is a practical computer vision problem with direct relevance to crop monitoring and precision agriculture, a domain where deep learning has been adopted broadly for disease detection, identification, and yield-related tasks [1]. Deep neural networks can achieve strong classification accuracy on curated leaf image datasets, but models with high parameter counts may be inconvenient for mobile or edge-oriented deployment, creating a tension between predictive accuracy and computational efficiency. A range of model compression techniques — including parameter pruning, quantization, and knowledge distillation — has been developed to ease this tension [2], and efficient architecture families such as the MobileNet line [3] were designed specifically for the resulting mobile and edge constraints.

This work studies that trade-off through knowledge distillation, in which a high-capacity teacher network guides the training of a lightweight MobileNetV3-Small student. Distillation is one of several routes to a smaller deployable model alongside post-hoc weight pruning and quantization [4], but it differs from those methods in shaping the student’s training directly rather than compressing an already-trained network; we revisit this distinction in Section V when we combine distillation with post-training quantization. The aim is not to claim deployment on a specific device, but to evaluate whether a compact student can retain a teacher’s predictive behavior while using far fewer parameters. Beyond the standard teacher-to-student comparison, we focus on a question that a single experiment cannot answer: does the *choice of teacher architecture* change how well the student learns?

We make three contributions: (i) a preliminary single-split study on 38-class PlantVillage confirming that distillation improves a compact student; (ii) a cross-validated study on

Cassava comparing three teacher architectures (ResNet50, EfficientNet-B2, ViT-B/16) under stratified five-fold cross-validation, with the teacher retrained per fold and paired significance tests against the baseline; and (iii) a quantization study showing that naive int8 quantization collapses the distilled student — unlike its teacher — that the failure is a property of the architecture rather than of distillation or initialization, and that it is localized to two identifiable blocks whose exemption (0.08% of parameters) restores essentially full accuracy post-training, outperforming quantization-aware training at a fraction of the cost. Along the way we document that the raw post-training accuracy of this architecture is so sensitive to the calibration draw that single-number int8 results for it are unsound.

II. RELATED WORK

PlantVillage [5] provides a widely used benchmark of labeled plant leaf images. Its clean imaging conditions are well suited to initial controlled experiments but less representative of field conditions. The Cassava leaf-disease dataset [6] covers five classes with strong class imbalance and real field imagery, providing a harder and more realistic testbed. Early work demonstrated high accuracy on PlantVillage with convolutional networks [7], but the literature has paid comparatively little attention to compressing these models for on-device inference.

Compressing a network for constrained hardware can be approached in several ways — pruning, quantization, low-rank factorization, and knowledge distillation — as surveyed by Cheng et al. [2]. This work examines the last two in sequence: we distil into a compact student and then attempt to quantize it, and we compare against pruning in Section V-G. Whether the two actually compose is not a foregone conclusion, and it turns out to be one of our findings (Section V-D).

Integer quantization for efficient inference is itself well studied. Jacob et al. [8] formalize integer-arithmetic-only inference and the fake-quantization scheme underlying quantization-aware training, and Krishnamoorthi [9] surveys the design space in detail. Two findings from that literature bear directly on our results. First, weight granularity matters: per-tensor quantization degrades depthwise-separable networks sharply, and per-channel scales are the standard remedy, precisely because a single extreme channel otherwise dictates the resolution of the rest [9]. Second, the MobileNet family is repeatedly singled out as the hard case for post-training quantization, which has motivated dedicated methods — weight equalization and bias correction to make the layers more amenable [10], and quantization-aware training where post-training methods fall short [11]. Our study is complementary to this line in three ways. We confirm the per-channel requirement on MobileNetV3-Small but show it is necessary and nowhere near sufficient; rather than a global reparameterization or a training-time fix, we *localize* the residual failure to two identifiable blocks and show that exempting them — a mixed-precision choice needing no retraining — recovers essentially full accuracy and outperforms quantization-aware training on

this model; and we quantify a calibration-draw instability that single-number int8 reports for this architecture class routinely omit.

Knowledge distillation [12] transfers information from a larger teacher to a smaller student by having the student match the teacher’s softened output probabilities, which encode inter-class similarity absent from one-hot labels. Later methods go beyond output logits and transfer intermediate representations, such as hidden-layer hints [13] and spatial attention maps [14]; a broad survey is given by Gou et al. [15]. We use the original logit-based formulation since our focus is the teacher architecture rather than the distillation mechanism, and a logit-based loss is the one formulation that applies unchanged across teachers whose internal representations are not comparable — a CNN feature map and a transformer patch-token sequence have no natural correspondence.

A recurring finding is that stronger teachers do not necessarily produce better students. Cho and Hariharan [16] show that a large capacity gap can impair distillation, and Mirzadeh et al. [17] propose intermediate teacher assistants to bridge it. Beyer et al. [18] argue that training consistency matters more than teacher accuracy. These results motivate our central question: when the student is fixed, does the teacher’s *architecture* — and not merely its accuracy — affect the distilled student? DeiT [19] distills *into* a transformer using a CNN teacher; our setting is complementary, distilling *from* a transformer and two CNN teachers into a convolutional student.

The three teachers span distinct architectural families: ResNet50 uses residual connections to ease optimization of deeper convolutional models [20]; EfficientNet-B2 applies compound scaling of depth, width, and resolution [21]; and ViT-B/16 processes images as patch sequences via self-attention [22]. The student in all experiments is MobileNetV3-Small [23], a compact architecture designed for mobile inference.

III. METHODOLOGY

Let z_t and z_s denote the teacher and student logits for an input image, and let y denote the ground-truth class label. Supervised teacher and supervised student baselines are trained with cross-entropy against the hard labels. For distillation, the teacher is first trained and then frozen, and the student is optimized using a weighted sum of the cross-entropy loss and a temperature-scaled Kullback–Leibler divergence term:

$$\mathcal{L} = \alpha T^2 \text{KL}\left(\text{softmax}\left(\frac{z_t}{T}\right) \parallel \text{softmax}\left(\frac{z_s}{T}\right)\right) + (1 - \alpha) \text{CE}(y, z_s). \quad (1)$$

The T^2 factor compensates for the gradient scaling introduced by softened probabilities. The temperature is fixed at $T = 4.0$ throughout; the distillation weight is $\alpha = 0.5$ in the PlantVillage study and $\alpha = 0.7$ in the main Cassava study. In both cases α is held constant across teachers so that any difference between teachers is attributable to the teacher rather than to the loss weighting.

IV. EXPERIMENTAL SETUP

All experiments use pretrained ImageNet weights, images resized to 224×224 pixels and normalized with ImageNet statistics, random horizontal flipping and small rotations during training, deterministic resizing for validation and test, and the best validation checkpoint for evaluation. All training runs, on both datasets, use a single NVIDIA RTX 4070 Laptop GPU (CUDA) with automatic mixed precision, so results are comparable across the two studies. Inference latency is treated separately in Section V-F, which reports that our hardware could not measure it reproducibly and therefore withholds most of the figures rather than publishing them.

A. Main Study: Cassava Cross-Validation

The main study uses the Cassava leaf-disease dataset (five classes) under stratified five-fold cross-validation. For each fold, the teacher is retrained from scratch on that fold’s training partition and then frozen before distilling the student; this avoids the bias that would arise if a single teacher had seen images that later appear in the test fold. The fold partition is fixed by seed, so every model sees identical folds. Three teachers are evaluated — ResNet50, EfficientNet-B2, and ViT-B/16 — each distilled into MobileNetV3-Small and compared against a supervised baseline; per-fold training schedules are given in Table I. Results are reported as mean $\pm$ standard deviation across the five folds, and each distilled student is compared against the baseline with a paired two-sided t -test on per-fold accuracies. All runs use an NVIDIA RTX 4070 Laptop GPU with automatic mixed precision.

The auxiliary analyses that require one fixed model rather than five — the post-training quantization study and the per-class error analysis — use a single 70/15/15 train/validation/test split of Cassava (14,979/3,209/3,209 images, seed 42) rather than the cross-validation folds.

On that split we retrain the supervised and distilled students under five training seeds each, varying only weight initialization and batch order. Both arms take exactly the budget used by the cross-validation student (5×10^{-4} , 15 epochs, batch 64), so the only difference between them is the loss, and the teacher — trained on the fixed split — remains valid for every replicate. *Every* single-model result we report — the quantization study, the per-class error analysis, the latency benchmark, and the seed-replicate comparison — is computed on one of these students, and wherever a single checkpoint is needed it is the seed-42 distilled replicate (float32 accuracy 84.51%). This matters more than it may appear. An earlier version of this work quantized a student trained for only 10 epochs; its float32 accuracy differed by 0.1 points but its int8 accuracy differed by more than ten, so no reader could have detected the substitution from the float32 numbers alone. Checkpoint provenance is not a bookkeeping detail when the quantity you are measuring is this sensitive to it.

B. Preliminary Study: PlantVillage Single Split

As a preliminary demonstration, a ResNet50 teacher, a supervised MobileNetV3-Small baseline, and a distilled student

TABLE I
PER-FOLD TRAINING CONFIGURATION FOR THE CASSAVA
CROSS-VALIDATION STUDY. ALL TEACHERS ARE DISTILLED INTO
MOBILENETV3-SMALL WITH $T = 4.0$ AND $\alpha = 0.7$.

Model	Role	Learning rate	Epochs	Batch
ResNet50	Teacher	3×10^{-4}	10	64
EfficientNet-B2	Teacher	3×10^{-4}	10	64
ViT-B/16	Teacher	1×10^{-4}	10	32
MobileNetV3-Small	Student / baseline	5×10^{-4}	15	64

are trained on all 38 classes of PlantVillage. We take the training partition of the public release (43,444 images) and re-split it into our own 70/15/15 train/validation/test partition (30,412/6,516/6,516 images, seed 42), so that the test images are held out from our own training rather than inherited from the distributed split. Training uses 15 epochs, batch size 32, $T = 4.0$, and $\alpha = 0.5$. Learning rates are 3×10^{-4} (teacher), 10^{-3} (supervised student), and 5×10^{-4} (distilled student). Both students are trained under three seeds, with the split held fixed so that only initialization and batch order vary; the teacher is trained once and reused, as it is unaffected by the student’s seed.

V. RESULTS

Experiments cover two datasets. On PlantVillage (38 classes, single split), we train one ResNet50 teacher and, under three training seeds each, a supervised MobileNetV3-Small baseline and a distilled student. On Cassava (five classes, five-fold CV), we train three teacher baselines, one student baseline, and three distilled students — one per teacher — plus an ablation over distillation hyperparameters, a set of post-training quantization studies (whole-model, per-block sensitivity, and mixed-precision, each replicated across the five distilled students), and a quantization-aware training run for each of those students (Section V-E). A separate batch-one latency benchmark (Section V-F) attempts to time every architecture on an idle machine, and reports which of those timings can be trusted. All metrics are macro-averaged precision, recall, and F1 alongside accuracy.

A. Cassava Cross-Validation

Table II summarizes the cross-validated results.

Distillation helps. All three teachers raise the student above the supervised baseline (81.67% accuracy) by a similar margin: +1.82 points for ResNet50 (83.49%), +1.45 for ViT-B/16 (83.12%), and +1.37 for EfficientNet-B2 (83.04%). In the paired t -test the ResNet50 and ViT-B/16 gains are statistically significant ($p = 0.029$ and $p = 0.036$); the EfficientNet-B2 gain does not reach significance ($p = 0.079$), most likely reflecting the limited power of five folds together with the larger fold-to-fold variance of its paired differences against the baseline (SD 1.31 points, versus 1.23 for ResNet50 and 1.05 for ViT-B/16).

The more notable result is how little the teacher matters. The three distilled students lie within 0.5 points of one

TABLE II
CROSS-VALIDATED TEST RESULTS ON CASSAVA (5-FOLD, MEAN $\pm$ STANDARD DEVIATION ACROSS FOLDS). PRECISION, RECALL, AND F1 ARE MACRO-AVERAGED. THE TEACHER IS RETRAINED PER FOLD; THE STUDENT AND KD HYPERPARAMETERS ARE FIXED.

Teacher	Student	Accuracy (%)	Precision (%)	Recall (%)	Macro-F1 (%)	Parameters
–	resnet50	85.13 $\pm$ 0.95	75.08 $\pm$ 1.40	73.69 $\pm$ 1.28	74.00 $\pm$ 0.78	23,518,277
–	efficientnet_b2	84.59 $\pm$ 0.67	73.91 $\pm$ 1.51	71.75 $\pm$ 0.86	72.60 $\pm$ 0.61	7,708,039
resnet50	mobilenet_v3_small	83.49 $\pm$ 0.31	72.70 $\pm$ 0.66	69.74 $\pm$ 2.62	70.72 $\pm$ 1.99	1,522,981
vit_b_16	mobilenet_v3_small	83.12 $\pm$ 0.72	72.90 $\pm$ 0.99	68.25 $\pm$ 2.36	69.85 $\pm$ 1.95	1,522,981
efficientnet_b2	mobilenet_v3_small	83.04 $\pm$ 0.41	71.75 $\pm$ 1.53	69.40 $\pm$ 1.98	70.34 $\pm$ 1.26	1,522,981
–	vit_b_16	82.40 $\pm$ 0.64	71.41 $\pm$ 1.28	69.08 $\pm$ 1.58	69.38 $\pm$ 1.26	85,802,501
–	mobilenet_v3_small	81.67 $\pm$ 1.01	69.83 $\pm$ 2.64	67.57 $\pm$ 1.38	68.22 $\pm$ 0.80	1,522,981

another (83.04–83.49%), a spread comparable to their across-fold standard deviations (0.31–0.72), even though the teachers differ substantially in accuracy (82.40–85.13%), architectural family (two CNNs and one transformer), and size (7.7M, 23.5M, 85.8M parameters). The ViT-B/16 student (83.12%) surpasses its own teacher (82.40%), which runs counter to the expectation that a very large teacher, separated from a 1.5M-parameter student by a wide capacity gap, would distill poorly. We interpret this teacher-agnostic behavior in Section VII.

B. Sensitivity to Distillation Hyperparameters

The main study fixes $T = 4.0$ and $\alpha = 0.7$ across teachers. These values were chosen *a priori*, before any sweep was run: $T = 4$ is the value used by Hinton et al. [12], and $\alpha = 0.7$ was set to weight the soft target above the hard label. The sweep below is therefore a post-hoc check that the fixed choice was reasonable, not the procedure that selected it — a distinction worth stating, because the sweep runs on fold 0, which is one of the five folds contributing to the headline cross-validated number, and had the defaults been *tuned* there the main result would inherit a leak.

To verify the choice is not an artifact, we run a single-split sensitivity analysis for the ResNet50 teacher on fold 0, sweeping $T \in \{1, 2, 4, 8\}$ at $\alpha = 0.7$ and $\alpha \in \{0.3, 0.5, 0.7, 0.9\}$ at $T = 4.0$. Because T and α enter only the student loss, the teacher is trained once and reused for every setting, so the sweep varies the distillation schedule alone. Table III reports the result.

The student is stable: accuracy stays within about 1.4 points over all settings (82.78–84.14%), with a gentle maximum at the default $T = 4.0$, $\alpha = 0.7$. Setting $\alpha = 0.9$ lowers macro-F1 to 69.39%, consistent with over-weighting the teacher at the expense of minority classes. The flat response confirms that the fixed configuration is well justified. These are single-split point estimates without fold error bars, and are intended as a sensitivity check rather than as a second cross-validated comparison.

C. PlantVillage Single Split

The main study establishes the teacher-agnostic effect on Cassava; a preliminary single-split experiment on the 38-class PlantVillage dataset checks that distillation helps at all on

a second, near-saturated task. Table IV reports the results. Both students are replicated over three training seeds on the fixed split, which matters here: a single run of this experiment is misleading, and in a way that happens to flatter neither conclusion.

Distillation raises the student from $97.84 \pm 0.59\%$ to $99.48 \pm 0.05\%$, a paired gain of $+1.64 \pm 0.64$ points ($p = 0.047$). The distilled student not only overtakes its supervised twin — same architecture, same parameter count, same budget, differing only in the loss — but matches the ResNet50 teacher itself (99.48%) at a fifteenth of its size.

The more interesting number is the standard deviation. Across seeds the supervised baseline has an SD of 0.59 points while the distilled student has 0.05 — a twelvefold smaller standard deviation. Distillation does not merely lift the student’s accuracy on this dataset, it very nearly removes its run-to-run variability, which is consistent with reading the soft-label term as a regularizer.

This asymmetry is also what makes a single run of the experiment treacherous, and the treachery is one-sided. All of the variance sits in the baseline: the three supervised seeds span 97.38–98.50%, while the three distilled seeds span 99.42–99.51%. Because the distillation gain is measured *against* that baseline, a single split estimates the gain only as well as it happens to draw the noisy cell — a lucky baseline halves the apparent gain, an unlucky one inflates it. On a near-saturated dataset the honest reading is that the direction is certain and the magnitude is not, which is why the quantitative claims of this paper rest on the cross-validated Cassava study.

D. Edge Compression: Quantization

We apply int8 post-training quantization (PTQ) on CPU to the distilled student described in Section IV — the seed-42 replicate, trained under the same schedule as every other model in this paper. *Dynamic* PTQ is applied in eager mode; *static* PTQ (weights and activations) uses the PT2E export-based API with the XNNPACK quantizer, calibrated on 1,024 training images. Because the calibration draw is itself a nuisance factor of the size we are trying to measure, each static configuration is run over five calibration seeds and reported as mean $\pm$ SD rather than as a single number. Table V reports the outcome.

TABLE III

SINGLE-SPLIT KD SENSITIVITY ANALYSIS FOR THE RESNET50 TEACHER (FOLD 0 OF THE CASSAVA CROSS-VALIDATION SPLIT). TEMPERATURE IS SWEEPED AT $\alpha = 0.7$ AND α AT $T = 4$; $\dagger$ MARKS THE DEFAULT USED IN THE MAIN STUDY. PRECISION, RECALL, AND F1 ARE MACRO-AVERAGED. VALUES ARE SINGLE-SPLIT POINT ESTIMATES (NO FOLD ERROR BARS).

Sweep	T	α	Accuracy (%)	Precision (%)	Recall (%)	Macro-F1 (%)
temperature	1	0.7	83.50	73.04	70.05	71.35
temperature	2	0.7	83.79	74.38	69.01	71.29
temperature	4	0.7 $\dagger$	84.14	73.99	70.29	71.52
temperature	8	0.7	83.67	73.26	70.03	71.37
alpha	4	0.3	82.78	70.59	70.76	70.61
alpha	4	0.5	83.60	72.91	69.64	70.89
alpha	4	0.7 $\dagger$	84.14	73.99	70.29	71.52
alpha	4	0.9	83.11	73.01	67.04	69.39

TABLE IV

TEST RESULTS ON THE 38-CLASS PLANTVILLAGE TASK (SINGLE 70/15/15 SPLIT, RTX 4070 LAPTOP GPU). THE TWO MOBILENETV3-SMALL ROWS ARE MEAN $\pm$ SD OVER THREE TRAINING SEEDS; THE TEACHER IS A SINGLE RUN. PRECISION, RECALL, AND F1 ARE MACRO-AVERAGED. NOTE THE ORDER-OF-MAGNITUDE DIFFERENCE IN THE STUDENTS’ STANDARD DEVIATIONS.

Model	Training strategy	Accuracy (%)	Precision (%)	Recall (%)	Macro-F1 (%)	Parameters
ResNet50	Teacher supervised	99.48	99.29	99.35	99.31	23,585,894
MobileNetV3-Small	Supervised baseline	97.84 $\pm$ 0.59	97.11 $\pm$ 0.53	96.78 $\pm$ 0.92	96.75 $\pm$ 0.73	1,556,806
MobileNetV3-Small	Distilled from ResNet50	99.48 $\pm$ 0.05	99.42 $\pm$ 0.08	99.19 $\pm$ 0.14	99.29 $\pm$ 0.10	1,556,806

Two properties of the tooling must be stated, because both are easy to misreport. First, PyTorch’s eager-mode dynamic quantization supports Linear and recurrent layers only: on a convolutional network it converts the classifier head and silently leaves every convolution in float32. Our “dynamic int8” row is therefore an int8 classifier head atop a float32 backbone — 2 Linear layers quantized, all 52 convolutions untouched — and we label it as such rather than as an int8 model. Second, the PT2E static path emits a reference-quantized graph that simulates int8 arithmetic with explicit quantize/dequantize operators in float. Its accuracy faithfully reflects int8 inference, but its serialized size and runtime do not, so we report no size or latency for the static rows.¹

Dynamic quantization is essentially lossless (84.51 $\rightarrow$ 84.48%) and shrinks the model 1.4 $\times$, but the caveat above explains why: it never touches the convolutional backbone, so the reduction is confined to the classifier head and the invariance of accuracy is unsurprising rather than a result. It is not a route to an int8 edge model. Nor would one expect it to buy speed, since the classifier head is not where the time is spent — though we are careful not to assert that as a measurement, because our hardware could not time this model reproducibly (Section V-F). Size and latency are distinct axes, and quantizing the head moves only the first. Our measurements use the `fbgemm` x86 backend; an ARM backend (`qnnpack`) would be the target for actual mobile deployment.

¹We initially obtained the static results through the legacy FX graph-mode API. That path is unsound for this model on torch 2.6: converting with the quantization configuration disabled everywhere — that is, quantizing nothing — still collapsed test accuracy from 84.2% to roughly 15% on the checkpoint then in use, so it measures a defect in the conversion pass rather than the effect of quantization. All static numbers reported here use the PT2E API.

Static quantization, which does quantize the convolutions, collapses the student from 84.5% to 35.5 $\pm$ 7.6% — far below the 62% obtained by always predicting the majority class, and therefore unusable. A sequence of controls establishes that this is a genuine property of the model rather than an artifact of our configuration, and locates the cause precisely.

It is not calibration-limited. Enlarging the calibration set eightfold leaves the student in the same collapsed regime: 28.6 $\pm$ 6.0% at 256 images, 35.5 $\pm$ 7.6% at 1,024, and 33.0 $\pm$ 5.8% at 2,048. The spread between those means (6.9 points) is no larger than the draw-to-draw noise within a single size ($\approx$ 6.5 points), so we cannot even resolve a size effect — and more to the point, no amount of calibration data rescues the model. We stress that this conclusion requires several draws per size: a single draw per size would have produced a 17-point spread that is pure noise, and would have invited exactly the wrong inference.

It is not the squeeze-and-excitation blocks. Holding every SE block in float32 while still quantizing the convolutions changes nothing: 35.97 $\pm$ 7.60% against 35.48 $\pm$ 7.63%, a difference an order of magnitude smaller than the noise on either. The SE blocks are the usual suspect for MobileNetV3 quantization failures, and here they are exonerated.

What does matter is weight granularity. Per-tensor quantization drops the student to 10.1 $\pm$ 0.2% — below the 20% chance rate, because the model degenerates into predicting essentially one class for every image. So per-channel weights are necessary, but nowhere near sufficient. This concerns *weight* outliers specifically: per-channel scales exist so that one extreme channel cannot dictate the resolution of the rest. Whether *activation* outliers explain the collapse is a separate and, as we show next, murkier question.

TABLE V

POST-TRAINING QUANTIZATION OF THE DISTILLED MOBILENETV3-SMALL ON THE CASSAVA SINGLE-SPLIT TEST SET. PRECISION, RECALL, AND F1 ARE MACRO-AVERAGED. STATIC ROWS USE PT2E REFERENCE QUANTIZATION, WHOSE SIMULATED GRAPH MAKES SERIALIZED SIZE UNREPRESENTATIVE OF A DEPLOYED INT8 MODEL (—). CHANCE IS 20% AND THE MAJORITY-CLASS RATE IS 62%. STATIC INT8 ACCURACIES ARE MEAN $\pm$ SD OVER *five calibration draws*: A SINGLE DRAW IS NOT A MEASUREMENT ON THIS ARCHITECTURE, SINCE THE DRAW ALONE MOVES ACCURACY BY SEVERAL POINTS.

Variant	Accuracy (%)	Precision (%)	Recall (%)	Macro-F1 (%)	Size (MB)
Float32 (baseline)	84.51	74.77	70.97	72.57	5.93
Dynamic int8 (classifier head only; 0/52 convs)	84.48	74.62	70.88	72.45	4.23
Static int8, per-channel weights	35.48 $\pm$ 7.63	47.21 $\pm$ 5.53	29.14 $\pm$ 2.39	20.25 $\pm$ 3.38	—
Static int8, per-channel, SE blocks in float32	35.97 $\pm$ 7.60	46.57 $\pm$ 5.05	29.44 $\pm$ 2.56	20.79 $\pm$ 3.66	—
Static int8, per-tensor weights	10.10 $\pm$ 0.24	12.72 $\pm$ 1.62	19.66 $\pm$ 0.17	7.26 $\pm$ 0.35	—

The failure is localized to two blocks. We quantize the student one block-group at a time, leaving everything else in float32, and repeat the whole sweep on each of the five distilled replicate students so that the per-block cost carries a standard deviation across training seeds (Table VI). From here on we report the five-student mean rather than the single seed-42 checkpoint profiled in Table V; under full int8 that checkpoint collapses to 35.5% and the five-student mean to 29.9%, the difference being seed variation of the kind the replication is designed to expose.² The collapse is not distributed at all. Two blocks carry essentially all of it: the input convolution `features.0` costs 57.4 $\pm$ 14.6 points on its own and the first bottleneck `features.1` costs 39.0 $\pm$ 5.9, whereas each of the remaining twelve groups costs at most 0.16 points. Quantizing `features.0` alone already lands the model in the same collapsed regime as quantizing the entire network ($\approx$ 27% versus $\approx$ 30%, equal within its seed-to-seed noise), so those two blocks account for the whole failure and the other twelve add nothing detectable. The gap between the two culprits and the rest spans three orders of magnitude, and the ordering is identical on all five students, even though the magnitude of the `features.0` effect itself varies substantially from seed to seed (40–71 points) — which is why it is reported with its spread rather than as a point estimate.

The mechanism resists a clean activation-level account. It is tempting to attribute the collapse to activation outliers, since per-tensor activation quantization is dominated by the largest value. One proxy supports this: comparing the largest per-convolution peak activation to the median across convolutions, the student scores 19.1 $\times$ against the teacher’s 5.7 $\times$, ranking the student as the more outlier-heavy. But that proxy is the exception, not the rule. Two other equally reasonable measures, both looking *within* each activation tensor rather than across convolutions, reverse the ranking: the within-tensor peak-to-median is 365 $\times$ for the teacher against 20 $\times$ for the

student, and a percentile-robust variant (peak over the 99.9th percentile) is 16.4 $\times$ for the teacher against 1.9 $\times$ for the student. On two of the three definitions it is the *teacher* — which quantizes losslessly — that is more outlier-dominated. When defensible measures of “outlier-dominated” disagree on direction, none is evidence, and we decline to rest an explanation on activation range. What we can say structurally is narrower: the two culprits are the first two blocks, and `features.0` in particular is the input convolution, whose error propagates through the entire network and which has only three input and sixteen output channels, leaving per-channel quantization almost nothing to average over. Why these two blocks and not others is a question our data localize sharply but do not fully explain, and we present it as such. Crucially, the remedy below does not depend on the mechanism.

Exempting the two blocks makes post-training quantization work. The localization is directly actionable. Keeping `features.0` and `features.1` in float32 while quantizing everything else recovers the student to 83.91 $\pm$ 0.27%, a drop of just 0.41 points from float32, replicated over the five students and five calibration draws (Table VII). The instability collapses with it — the standard deviation falls from 13.21 to 0.27 — because the two blocks that carried the calibration-draw sensitivity are no longer quantized. Crucially, this costs almost nothing: the exempted blocks hold 1,208 of the student’s 1,522,981 parameters, or 0.08%, so the mixed-precision model is within 0.004MB of the fully int8 one. Exempting only `features.0` is not enough (43.4 $\pm$ 6.4%); both blocks are required. Post-training quantization, in short, is usable on this architecture — but only if the input convolution and first bottleneck are left in float, and standard tooling does neither by default nor makes it easy to do by hand.

The result is a property of the architecture, not of Cassava. We repeat the comparison on the 38-class PlantVillage student of Section V-C — the same MobileNetV3-Small architecture, a very different dataset and class count. The pattern is identical and, if anything, cleaner: full int8 collapses the student from 99.51% to 37.68 $\pm$ 2.18%, exempting `features.0` alone recovers only partially (43.47 $\pm$ 1.56%), and exempting both blocks restores it to 99.43 $\pm$ 0.03% — a drop of 0.08 points. The two culprit blocks, and the mixed-precision remedy, therefore transfer across datasets unchanged, which is what

²Isolating a single block correctly is more delicate than it appears. The obvious construction — disable quantization globally, then re-enable it by module name for the target block — silently annotates *nothing* for three of the fourteen groups, including the input convolution, so those blocks come back bit-identical to float32 and would be misreported as costing zero. We instead annotate globally (the path the full-model result uses) and strip the annotations outside the target block, and we assert that each isolated model contains a non-zero number of quantize operators before trusting its accuracy.

TABLE VI

PER-BLOCK SENSITIVITY, REPLICATED OVER THE FIVE DISTILLED STUDENTS ($\times$ THREE CALIBRATION DRAWS EACH); VALUES ARE DROP FROM FLOAT32, MEAN $\pm$ SD ACROSS STUDENTS. EACH ROW QUANTIZES *only* THAT BLOCK-GROUP. THE COLLAPSE IS LOCALIZED: TWO BLOCKS CARRY IT, THE OTHER TWELVE ARE INDIVIDUALLY HARMLESS. THE TWO CULPRITS AND THE THREE NEXT-LARGEST ARE SHOWN; THE REMAINING NINE GROUPS ARE OMITTED (EACH ≤ 0.10 POINTS). `FEATURES.0` ALONE EXCEEDS THE ALL-GROUPS DROP BECAUSE BOTH LAND NEAR THE CHANCE FLOOR, WHERE THE MEASURE SATURATES.

Quantized group (only)	Drop from float32 (pts)
<code>features.0</code> (input conv)	57.4 ± 14.6
<code>features.1</code> (first bottleneck)	39.0 ± 5.9
<code>features.4</code>	0.16 ± 0.22
<code>features.7</code>	0.10 ± 0.14
<code>features.3</code>	0.09 ± 0.15
All groups (full int8)	54.5

TABLE VII

MIXED-PRECISION POST-TRAINING QUANTIZATION ON THE CASSAVA TEST SET, OVER FIVE DISTILLED STUDENTS $\times$ FIVE CALIBRATION DRAWS (MEAN $\pm$ SD ACROSS STUDENTS). KEEPING THE TWO SENSITIVE BLOCKS IN FLOAT32 RECOVERS FULL-PRECISION ACCURACY AT 0.08% OF PARAMETERS, WITH NO RETRAINING.

Configuration	Int8 accuracy (%)	Gap vs. float32
Full int8 (all blocks)	29.86 ± 13.21	-54.5
Exempt <code>features.0</code>	43.42 ± 6.36	-40.9
Exempt <code>features.0</code> + <code>features.1</code>	83.91 ± 0.27	-0.41

one expects if the cause is the architecture’s first layers rather than any property of the Cassava data.

The transfer to PlantVillage already implicates the architecture over any property of the Cassava data. A within-dataset replication rules out the one remaining alternative — that the collapse is an accident of the particular optimum *this* student reached — and, as a byproduct, gives us the material to characterize the instability the int8 numbers carry. We quantize both student arms across five training seeds. The data split is held fixed, so the teacher never sees a test image and only the student’s initialization and batch order change, and both arms use identical budgets (5×10^{-4} , 15 epochs, batch 64): the sole difference between them is the loss. Each of the ten resulting students is quantized through the same PT2E path. Table VIII reports the outcome.

Initialization is not the cause either. Every one of the ten students collapses: int8 accuracy spans 12.8–49.4% against a float32 range of 81.5–84.8%, and not one comes close to the 62% majority-class rate. The supervised students — never distilled, and therefore different initializations converging to different optima — collapse just as the distilled ones do, so neither distillation nor the student’s initialization can account for the failure. Meanwhile the ResNet50 teacher passes through the identical pipeline losslessly: over five calibration

draws it scores $86.65 \pm 0.17\%$ against a float32 86.51% ,³ a difference indistinguishable from zero. This confirms both that the quantization path is sound and that the failure is specific to the compact architecture.

The replication further shows that post-quantization accuracy is too unstable to support any finer claim, and we report this explicitly because it governs how the int8 numbers in this paper should be read. Two independent sources of noise act on it. Across training seeds, int8 accuracy varies with a standard deviation of 9.0 points (supervised) and 13.2 points (distilled), even though the corresponding float32 accuracies vary by less than a point. On top of that, the calibration draw alone moves the result: holding a single checkpoint completely fixed and varying only which 1,024 augmented training images are used for calibration, the distilled student’s int8 accuracy ranges over 27.6–44.2% (35.5 ± 7.6). A single int8 figure is therefore one sample from a wide distribution over two nuisance factors, and we fix both the training and calibration seeds so that the values we report reproduce.

This instability is itself architecture-specific, and the contrast is stark. Subjected to the same five calibration draws, ResNet50’s int8 accuracy has a standard deviation of 0.17 points against the student’s 7.63 — a factor of forty-five. MobileNetV3-Small does not merely quantize badly; the very *measurement* of how badly it quantizes is unstable, whereas for ResNet50 post-training quantization is both lossless and repeatable. Sensitivity to the calibration set is thus a second symptom of the same underlying cause, and it suggests that reporting int8 accuracy for depthwise-separable architectures without quantifying calibration variance is unsound practice.

This has a direct consequence. On one seed the distilled student retained far more accuracy under quantization than the supervised one, which invites the conclusion that distillation confers quantization robustness. Over five paired seeds that gap is $+5.5 \pm 17.5$ points and not significant (paired t -test, $p = 0.52$); across repeated measurements its point estimate wandered between $+1.6$ and $+10.0$ points without ever approaching significance. **We therefore make no claim that distillation aids quantization.** By contrast, the same replication reproduces the *float32* benefit of distillation exactly and stably ($+1.66 \pm 0.95$ points, $p = 0.017$) under a fixed split and a matched budget, corroborating the cross-validated result of Section V-A by an independent route. The contrast is instructive: distillation’s effect on float accuracy is small but highly reproducible, whereas its apparent effect on quantized accuracy is large but pure noise.

All of this is consistent with MobileNetV3-Small being the one architecture in its family for which `torchvision` ships no post-training quantized variant, while shipping one for ResNet50 and for MobileNetV3-Large.

³The teacher’s training log records 86.54% for this same checkpoint. The evaluation is not bit-wise deterministic across devices — that run scored it on the GPU, the quantization study re-scores it on the CPU — and the two differ by a single image out of 3,209. We quote the CPU figure throughout, since it is the one measured under the same conditions as the quantized models it is being compared against.

TABLE VIII

IS THE INT8 COLLAPSE CAUSED BY OUR STUDENT, OR BY ITS ARCHITECTURE? THE TWO MOBILENETV3-SMALL ARMS ARE REPLICATED OVER FIVE TRAINING SEEDS ON A FIXED SPLIT WITH MATCHED TRAINING BUDGETS, SO ONLY THE LOSS DIFFERS. EVERY MODEL HERE — STUDENTS AND TEACHER ALIKE — IS QUANTIZED OVER THE *same* FIVE CALIBRATION DRAWS AND AVERAGED, SO THE STUDENT SDs ARE VARIATION ACROSS TRAINING SEEDS AND NOT CALIBRATION NOISE; THE TEACHER IS A SINGLE CHECKPOINT, SO ITS SD IS CALIBRATION NOISE ALONE. ALL MODELS ARE QUANTIZED THROUGH THE IDENTICAL PT2E PATH (PER-CHANNEL WEIGHTS, 1,024 CALIBRATION IMAGES) AND EVALUATED ON THE CASSAVA SINGLE-SPLIT TEST SET. THE INT8 STANDARD DEVIATIONS ARE LARGE BY NATURE, NOT BY ACCIDENT: SEE THE TEXT ON CALIBRATION NOISE.

Model	Float32 (%)	Int8 (%)	Paired p (int8)
MobileNetV3-Small, supervised (5 seeds)	82.66 ± 0.66	24.32 ± 8.99	
MobileNetV3-Small, distilled (5 seeds)	84.32 ± 0.36	29.86 ± 13.21	0.52 (n.s.)
ResNet50, teacher (5 calib. draws)	86.51	86.65 ± 0.17	–

E. Quantization-Aware Training

The exemption above is one route to an int8 student, and because it needs no retraining it is the cheaper one. Quantization-aware training (QAT) is the standard alternative: it quantizes the *whole* network — including the two sensitive blocks — but inserts fake-quantization nodes during a fine-tuning phase so the weights adapt to the quantization error rather than merely suffering it. We ran it, both to see how far it closes the gap and to compare it against the exemption.

We fine-tune each of the five distilled students with the same XNNPACK per-channel quantizer, the same fixed split, and the same test set used throughout, keeping the teacher in the loop so the distillation signal is not discarded, for 30 epochs with a cosine-annealed learning rate. Model selection is on the *converted* int8 model, not on the fake-quantized proxy, because those two can disagree and only the former is what ships.

QAT recovers the collapse, and removes the instability. It lifts the fully-quantized student from $29.86 \pm 13.21\%$ to $80.69 \pm 0.43\%$ (Table IX), an improvement of 51 points from far below the 62% majority-class rate to comfortably above it. The standard deviation falls 31-fold, and the calibration-draw noise vanishes entirely — not because we controlled it but because QAT has no calibration set to draw: the observers learn their ranges from the training data during fine-tuning. The irreproducibility documented above is thus a property of *post-training* quantization on this architecture, not of int8 inference on it.

The number is converged, which took some care to establish. Our first QAT run (5 epochs, learning rate 10^{-5}) reached $74.56 \pm 1.08\%$ and a second (12 epochs, 5×10^{-5}) reached $79.19 \pm 0.81\%$; both were still improving when the budget ran out, and either would have been an under-trained number reported as a limit. The 30-epoch schedule we report does settle: its per-epoch validation trace, released with the results, is flat from about epoch 12 onward (seed 42 oscillates between 79.1 and 80.7% over its last six evaluations while the learning rate anneals to zero), so the reported figure is a plateau rather than a stopping point, and this can be checked

TABLE IX

TWO ROUTES TO AN INT8 STUDENT, ON THE SAME FIVE DISTILLED STUDENTS AND THE SAME SPLIT (MEAN $\pm$ SD OVER TRAINING SEEDS). MIXED-PRECISION PTQ KEEPS TWO BLOCKS (0.08% OF PARAMETERS) IN FLOAT AND NEEDS NO RETRAINING; QAT QUANTIZES EVERYTHING BUT REQUIRES A FINE-TUNING PHASE. THE CHEAPER METHOD IS ALSO THE MORE ACCURATE ONE.

Route	Retraining	Accuracy (%)	vs. float32
Full int8, post-training	none	29.86 ± 13.21	–54.5
Quantization-aware training	30 epochs	80.69 ± 0.43	–3.6
Mixed-precision PTQ (exempt 2 blocks)	none	83.91 ± 0.27	–0.41

rather than taken on trust.

But mixed-precision PTQ is the better recipe. The comparison is one-sided. Exempting two blocks recovers $83.91 \pm 0.27\%$ with no retraining at all; QAT, after thirty epochs of fine-tuning through the whole network, reaches only $80.69 \pm 0.43\%$ — 3.2 points *lower*, for far more work. QAT’s sole advantage is that every layer is int8, against 99.92% of the layers under the exemption; on this student that buys nothing but costs three points of accuracy and a training loop. The residual gap to float32 tells the same story: 0.41 points for mixed-precision PTQ against 3.6 for QAT. Neither is free — int8 deployment on this architecture does cost accuracy — but the cheaper method is also the more accurate one. QAT would be the tool of choice only where the target runtime cannot execute a handful of float operations at all; whether combining the two — QAT with the input blocks exempted — closes the residual entirely is a question we leave open, having not run it.

F. Inference Latency

Parameter count is the compression metric used above, but it is a proxy, and the quantity that matters at the edge is time per image. We attempted a batch-one benchmark on an idle machine — 300 warm-up passes, 300 timed passes, the whole measurement repeated five times from scratch, medians rather than means — and required each figure to satisfy a stability criterion fixed in advance: the medians of the five independent repeats must agree to within 10%. Timings that fail this criterion are not reported.

The criterion is applied *across* the five executions, not within one. This distinction is the methodological point of the section. Each execution already reports the spread of its own five internal repeats, and that in-run figure is badly misleading: rows that pass it comfortably still move by tens of percent between one execution of the benchmark and the next. ResNet50 on CPU passed its in-run check on an early attempt at 92.6 ms and passed it again, just as comfortably, at 65.9 ms on a later one. Only re-executing the whole benchmark from a cold start exposes this, and an earlier attempt of ours — taken minutes after several hours of GPU training — was contaminated by exactly the residual thermal state that the in-run check cannot see. We discarded it and re-measured on a machine idled and cooled beforehand (external CPU load below 5%, verified throughout each run).

TABLE X

BATCH-ONE LATENCY, REPORTED AS THE RANGE OVER FIVE INDEPENDENT EXECUTIONS OF THE ENTIRE BENCHMARK ON A COOLED, IDLE MACHINE. A FIGURE IS PUBLISHED ONLY IF THOSE FIVE MEDIAN AGREE TO WITHIN 10%, A CRITERION FIXED IN ADVANCE; ROWS THAT FAIL ARE WITHHELD (—) RATHER THAN REPORTED WITH A CAVEAT. THE GPU IS AN RTX 4070 LAPTOP — A MOBILE PART, NOT THE DESKTOP 4070.

Model	Params	CPU (ms)	GPU (ms)
ResNet50 (teacher)	23.5M	65.9–69.8	—
EfficientNet-B2 (teacher)	7.7M	—	—
ViT-B/16 (teacher)	85.8M	108.9–116.4	6.03–6.15
MobileNetV3-Small (student)	1.5M	12.61–12.72	3.74–3.95
MobileNetV3-Small, dynamic int8	1.5M	—	n/a

Four of the nine rows still fail and are withheld (Table X). But five survive, and they support the claims below. **The student’s CPU latency is the most reproducible figure in the benchmark** (12.6–12.7 ms across five executions, a 0.8% spread), and against the teachers on the same device it gives a genuine speed-up: $5.3\times$ over ResNet50 (67.3 ms) and $8.9\times$ over ViT-B/16 (112.7 ms). This is the edge-relevant comparison, since a deployed student runs on a CPU, not on a datacentre GPU.

But a $15\times$ compression is not a $15\times$ speed-up. The measured CPU gain is $5.3\times$, and on the GPU the advantage nearly evaporates: at batch one the student takes 3.8 ms against ViT-B/16’s 6.1 ms, a mere $1.6\times$ for a model with $56\times$ fewer parameters, because kernel-launch overhead, not arithmetic, sets the floor. **Parameter count is in fact a poor proxy for GPU latency:** EfficientNet-B2 (7.7M) is slower than ViT-B/16 (85.8M) on the GPU in every one of the five executions, and by a margin whose ranges do not overlap (7.9–8.7 ms against 6.0–6.2 ms) even though EfficientNet-B2’s point estimate is not itself stable enough to report. Ordering can be robust where magnitude is not.

The same reasoning settles the quantization question raised in Section V-D. Dynamic int8 quantization of the student produces no speed-up whatsoever: its point estimate fails our criterion and is withheld, but across all five executions *every* int8 measurement (12.79–14.82 ms) was slower than *every* float32 measurement (12.61–12.72 ms). The int8 classifier head buys a $1.4\times$ smaller file and, on this hardware, not a single millisecond.

We withhold rather than caveat, and one case shows why that discipline matters. ResNet50’s GPU median landed at 4.1, 4.4, 4.2, and 4.3 ms on four executions and at 7.8 ms on the fifth. Dropping the fifth would yield a tidy, publishable row — and would be exactly the selective reporting this section exists to avoid — so the row is withheld. The broader lesson for the field is that a latency number obtained from a single execution of a benchmark on a thermally constrained machine is not a measurement, however many internal repeats it averages over; only re-running the benchmark itself reveals whether the number is real.

G. Comparison with the Literature

To situate our absolute numbers, Table XI places them beside published results on the same two datasets. Evaluation protocols differ — Mohanty et al. use an 80/20 split, Liu et al. a 90/10 split, and we use stratified five-fold cross-validation — so the comparison controls for the dataset, and approximately for the architecture, but not for the evaluation protocol.

On PlantVillage the distilled student reaches $99.48 \pm 0.05\%$ accuracy, above the 99.35% of Mohanty et al.’s GoogLeNet [7] and approaching the 99.75% of Too et al.’s DenseNet-121 [24]. The dataset is close to saturated at this accuracy, so the meaningful axis is model size. Pruning is the natural point of comparison: in a separate study, Too et al. [25] prune DenseNet for exactly this purpose, and their best pruned model retains 99.24% on PlantVillage while reducing parameters by about 14% and FLOPs by about 25%. Our distilled student attains a comparable 99.48% with 1.56M parameters. Relative to the unpruned DenseNet-121 ($\approx 7.0M$), that is a $4.5\times$ reduction, against the $1.16\times$ that filter pruning achieves — so distilling into an architecture designed for mobile inference compresses roughly four times more aggressively than pruning a large backbone, at no cost in accuracy. The two are complementary rather than competing, and the protocols are not identical, so this is an indication rather than a controlled comparison.

The Cassava comparison is the more informative one, because Liu et al. [26] evaluate single models on the *same* Kaggle Cassava Leaf Disease Classification release [6] that we use — the same five classes, collected in Uganda by the National Crop Resources Institute and the Makerere University AI Laboratory — so their per-architecture numbers are comparable to ours. They report 85.2% for ResNet50 and 84.1% for “MobileNet v3”, against our $85.13 \pm 0.95\%$ ResNet50 teacher and $83.49 \pm 0.31\%$ distilled MobileNetV3-Small student. The ResNet50 comparison is exact. The MobileNetV3 one is not: Liu et al. do not state which variant they use and report no parameter counts, and since MobileNetV3-Large is the more common default their figure may well correspond to the larger model, in which case our Small student is achieving a comparable score with roughly a third of the parameters. We therefore read this row as indicative rather than as a like-for-like match. Either way our models land within about a point of independently published results on the same data, which supports the correctness of our training pipeline and indicates that the $\sim 84\%$ regime is simply what single models attain on this dataset rather than a shortfall of our setup. Stronger results exist: the multi-scale fusion model of Liu et al. attains 88.1%, the winning entry of the associated Kaggle competition reached 91.32% on the private leaderboard, and the best reported result we are aware of is the 91.43% of Che et al. [27]. These, however, rely on ensembles, test-time augmentation, object segmentation, and considerably longer training schedules, all of which would confound the controlled teacher-architecture comparison that is the objective of this work.

TABLE XI

COMPARISON WITH PUBLISHED RESULTS ON THE SAME DATASETS. THE CASSAVA ROWS OF LIU ET AL. USE THE SAME KAGGLE RELEASE AND THE SAME FIVE CLASSES AS OURS, SO THE ARCHITECTURES ARE DIRECTLY COMPARABLE; THE EVALUATION PROTOCOL STILL DIFFERS (THEIR 90/10 SPLIT VS. OUR STRATIFIED FIVE-FOLD CROSS-VALIDATION), AS DOES MOHANTY ET AL.’S 80/20 PLANTVILLAGE SPLIT. LITERATURE VALUES ARE AS REPORTED BY THE CITED AUTHORS. OUR CASSAVA ROWS ARE MEAN $\pm$ SD OVER FIVE FOLDS; OUR PLANTVILLAGE STUDENT ROW IS MEAN $\pm$ SD OVER THREE TRAINING SEEDS ON A SINGLE SPLIT. PARAMETER COUNTS MARKED $\approx$ ARE THE STANDARD IMAGENET VARIANTS OF THE CITED ARCHITECTURES, ADJUSTED FOR THE 38-CLASS HEAD; TOO ET AL. [25] REPORT THEIR PRUNING AS A RELATIVE REDUCTION RATHER THAN AN ABSOLUTE COUNT. OUR PARAMETER COUNTS DIFFER SLIGHTLY BETWEEN THE TWO DATASETS BECAUSE THE CLASSIFIER HEAD IS SIZED TO THE NUMBER OF CLASSES (38 FOR PLANTVILLAGE, 5 FOR CASSAVA); THE BACKBONES ARE IDENTICAL.

Work	Model	Accuracy (%)	Parameters
<i>PlantVillage (38 classes)</i>			
Mohanty et al. [7]	GoogLeNet	99.35	$\approx 6.8\text{M}$
Too et al. [24]	DenseNet-121 (unpruned)	99.75	$\approx 7.0\text{M}$
Too et al. [25]	DenseNet (pruned)	99.24	$\approx 14\%$ fewer
Ours	ResNet50 (teacher)	99.48	23.6M
Ours	MobileNetV3-Small (distilled)	99.48 ± 0.05	1.56M
<i>Cassava (5 classes)</i>			
Liu et al. [26]	MobileNetV3	84.1	—
Liu et al. [26]	ResNet50	85.2	—
Liu et al. [26]	EfficientNet-B6	86.5	—
Liu et al. [26]	MSFM (proposed, multi-scale)	88.1	—
Ours	ResNet50 (teacher)	85.13 ± 0.95	23.5M
Ours	MobileNetV3-Small (baseline)	81.67 ± 1.01	1.5M
Ours	MobileNetV3-Small (distilled)	83.49 ± 0.31	1.5M

TABLE XII

PER-CLASS PERFORMANCE OF THE DISTILLED MOBILENETV3-SMALL ON THE CASSAVA SINGLE-SPLIT TEST SET.

Class	Precision	Recall	F1	Support
Bacterial blight	0.642	0.497	0.560	155
Brown streak disease	0.772	0.707	0.738	307
Green mottle	0.773	0.693	0.731	378
Healthy	0.626	0.701	0.662	385
Mosaic disease	0.925	0.951	0.938	1,984

VI. ERROR ANALYSIS

The gap between accuracy and macro-F1 throughout the Cassava results is a direct consequence of class imbalance. On the single-split test set, *mosaic disease* accounts for 1,984 of 3,209 images (62%), whereas *bacterial blight* has only 155 (5%). Table XII and Figure 1 give the per-class performance and confusion matrix of the distilled student.

The student performs well on the dominant class (*mosaic disease*, F1 0.94) but degrades on minority classes, reaching its worst on *bacterial blight* (F1 0.56, recall 0.50). Because accuracy is dominated by the majority class while macro-F1 weights all classes equally, accuracy (84.5%) substantially overstates per-class competence (macro-F1 72.6%), which is why we report both throughout. The confusion matrix reveals two systematic error patterns. First, minority diseases are frequently predicted as *healthy*: 51 of 155 bacterial-blight leaves (33%) and 38 brown-streak leaves are missed this way. These are false negatives — diseased plants reported as healthy — which is the costliest error type in crop monitoring, and the fact that they concentrate in the rare classes means the headline accuracy conceals them almost entirely. Second, the two visually similar mottling classes, *green mottle* and *mosaic disease*, are mutually confused (67 and 29 cases),

reflecting genuine visual similarity rather than imbalance alone. Distillation improves the student overall but inherits this confusion structure rather than resolving the hardest minority cases; addressing them would likely require class re-balancing, targeted augmentation, or cost-sensitive training.

A t-SNE projection of the student’s penultimate embeddings (Figure 2) tells the same story geometrically: *mosaic disease* forms a large, well-separated cluster, consistent with its 0.94 F1, whereas *bacterial blight*, *green mottle*, and *healthy* overlap in a shared region — precisely the minority classes the confusion matrix shows being conflated. The representation is therefore well organized for the majority class but only weakly discriminative for the rare ones.

VII. DISCUSSION

The central observation is that the distilled student is largely insensitive to its teacher: across a residual CNN, a compound-scaled CNN, and a vision transformer — differing widely in accuracy, architecture, and size — the three students fall within 0.5 points of one another, and the weakest and largest teacher (ViT-B/16) distills as effectively as the strongest. This points to the softened-label regularization of distillation, rather than the teacher’s capacity or architecture, as the source of the 1.4–1.8-point gain. The implication for edge settings is convenient: a small, inexpensive teacher suffices. The latency benchmark adds a caveat to the compression story, however. The student is genuinely faster where it matters — $5.3\times$ over ResNet50 on CPU, the device an edge deployment actually uses — but that is far short of the $15\times$ parameter reduction, and on a GPU at batch one it shrinks to $1.6\times$. Parameter count is not even monotonically related to latency: EfficientNet-B2 has a third of ResNet50’s parameters, and less than a tenth of ViT-B/16’s, yet is slower than ViT-B/16 on the GPU in every execution we ran. Compression ratios and speed-ups are different currencies, and

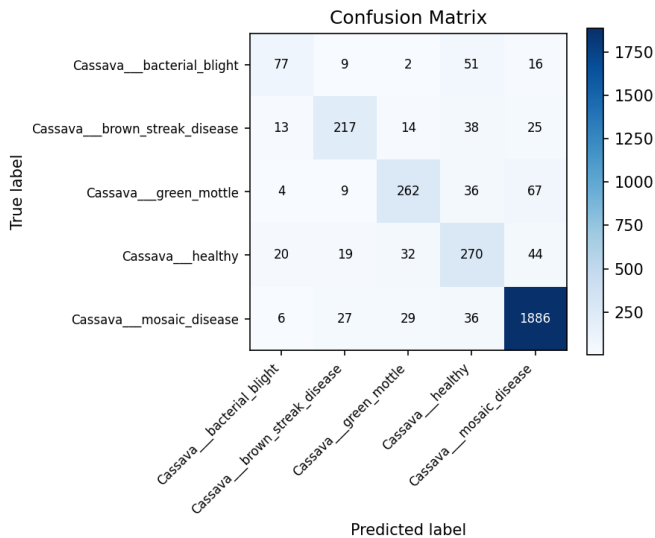


Fig. 1. Confusion matrix of the distilled MobileNetV3-Small on the Cassava single-split test set. Errors concentrate in the minority classes.

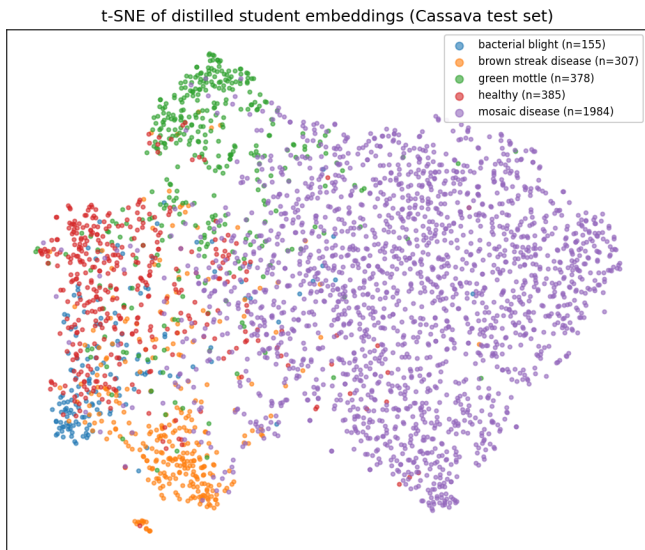


Fig. 2. t-SNE projection of the distilled student’s 1024-dimensional penultimate embeddings on the Cassava single-split test set. The majority *mosaic disease* class separates cleanly, while the minority classes overlap, mirroring the per-class scores (Table XII) and the confusion matrix (Figure 1).

papers in this area — ours included, before we tried to measure it — routinely quote the first while implying the second.

Compression, by contrast, is where the compact student proves brittle — until the brittleness is localized. Naive int8 quantization of the convolutions destroys the model, and the cause is the architecture rather than our student: the same pipeline quantizes the ResNet50 teacher losslessly and repeatably ($86.65 \pm 0.17\%$), while ten MobileNetV3-Small students spanning five seeds and both training objectives collapse without exception. But the failure is not diffuse. Quantizing the network one block at a time places almost

all of it on two blocks — the input convolution and the first bottleneck — and exempting just those two, a mere 0.08% of the parameters, restores essentially full accuracy (83.9%, within about 0.4 points of float32) with no retraining; the same exemption transfers unchanged to the 38-class PlantVillage student, so it is a property of the architecture rather than of one dataset. Distillation and post-training quantization *do* compose, then, but only once the two sensitive blocks are held out; applied naively they do not, which is what makes standard tooling misleading here, since it neither exempts those blocks by default nor makes doing so straightforward. Quantization-aware training reaches a similar place by a costlier road — thirty epochs of fine-tuning for 80.7%, three points *below* the exemption — so it is the fallback, not the recommended route.

We explicitly do *not* claim that distillation improves quantizability: a single pair of checkpoints suggested it did, but replication over five seeds showed the apparent advantage to be within the noise of an extremely unstable quantity ($p = 0.52$). That instability is itself worth reporting. Naive post-training-quantization accuracy on this architecture is not a property of the trained model alone — it also depends on which images happen to be drawn for calibration, to the tune of ± 8 points on fixed weights, a sensitivity that itself lives almost entirely in the two culprit blocks and vanishes once they are exempted. Studies that report a single int8 number for a MobileNetV3-class model, ourselves included until we checked, are reporting one sample from a wide distribution. What remains true regardless of method is that int8 deployment is a real trade rather than a free lunch — even the best route costs 0.4 points — and that the consequential architectural choice for a quantized deployment is the *student*, not the teacher, an inversion of where this work began.

Several limitations qualify these conclusions. Distillation hyperparameters are held fixed across teachers, so a teacher-specific T or α might widen the currently negligible gaps. With only five folds, statistical power is limited, as the borderline EfficientNet-B2 result illustrates; our claim is therefore that any teacher effect is *small*, not that it is exactly zero, and more folds or repeated runs would tighten the bound. Retraining every teacher on every fold is what licenses that claim — it is the only way to keep a teacher from having seen images that later appear in its own test fold — but it multiplies training cost by the number of folds, which is why the comparison is limited to three teachers. Finally, both datasets are curated, so accuracy on field images with cluttered backgrounds and varying lighting may be lower. Our latency evidence is also partial: on a thermally constrained laptop, four of the nine model/device timings we attempted were not reproducible across independent executions of the benchmark and are withheld (Section V-F), and the timings we do report come from a desktop CPU and a mobile GPU rather than from an ARM edge target. Re-measuring on the deployment device itself — where the $5.3\times$ CPU speed-up we observe may look quite different — is the first thing we would do next. The PlantVillage study uses a single split rather than cross-validation; we mitigate this by replicating both students over

three seeds, which is what revealed that its supervised baseline — not its distilled student — carries essentially all of the run-to-run variance on that near-saturated dataset.

VIII. CONCLUSION

We studied knowledge distillation from large teachers into a compact MobileNetV3-Small student for edge-efficient plant disease classification. On PlantVillage, a ResNet50 teacher distills into a student with $99.48 \pm 0.05\%$ accuracy at about $15\times$ fewer parameters, and with a twelvefold smaller seed-to-seed standard deviation than the same student trained supervised — distillation stabilizes the compact model as well as improving it. The main cross-validated study on Cassava showed that distillation raises the student by 1.4–1.8 accuracy points over a supervised baseline, yet the choice of teacher architecture — ResNet50, EfficientNet-B2, or ViT-B/16 — had little effect, indicating the benefit stems from the distillation procedure itself rather than from the teacher. This is encouraging for deployment, since a small, cheap teacher suffices. We are careful about speed, though: on a thermally constrained laptop only five of nine batch-one timings were reproducible across independent runs of the benchmark, and we withhold the rest. Those that survive show the student is genuinely faster where it matters — $5.3\times$ over ResNet50 on CPU — but far short of its $15\times$ parameter reduction, and only $1.6\times$ on the GPU, where among the teachers the smallest model is in fact the slowest. A parameter count is not a speed-up in disguise. Naive int8 quantization of the distilled student, however, collapses it, and control experiments locate the cause in the MobileNetV3-Small architecture rather than in distillation: the ResNet50 teacher quantizes losslessly through the same pipeline, while ten students spanning five seeds and both training objectives collapse without exception. A per-block sweep then shows the failure is not diffuse but localized to two blocks — the input convolution and the first bottleneck — which alone cost 57 and 39 accuracy points while the other twelve cost at most 0.2 each. This is directly actionable: keeping those two blocks (0.08% of the parameters) in float32 makes post-training quantization essentially lossless (83.9%, within about 0.4 points of float32) with no retraining, and outperforms the quantization-aware training that a naive reading of the collapse would have prescribed (80.7% after thirty fine-tuning epochs).

The practical recipe that emerges is therefore both narrower and cheaper than the one we set out to validate. Distil — from whichever teacher is cheapest, since the architecture of the teacher barely matters — and then quantize post-training with the input convolution and first bottleneck held in float. Int8 deployment is not free even so; the best route we found still costs 0.4 points against full precision. But it is reachable without retraining, provided the two sensitive blocks are identified and excluded — something standard tooling does not do on its own.

REFERENCES

- [1] A. Kamilaris and F. X. Prenafeta-Boldú, “Deep learning in agriculture: A survey,” *Computers and Electronics in Agriculture*, vol. 147, pp. 70–90, 2018.
- [2] Y. Cheng, D. Wang, P. Zhou, and T. Zhang, “Model compression and acceleration for deep neural networks: The principles, progress, and challenges,” *IEEE Signal Processing Magazine*, vol. 35, no. 1, pp. 126–136, Jan. 2018.
- [3] M. Sandler, A. Howard, M. Zhu, A. Zhmoginov, and L.-C. Chen, “MobileNetV2: Inverted residuals and linear bottlenecks,” in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. IEEE, 2018, pp. 4510–4520. [Online]. Available: https://openaccess.thecvf.com/content_cvpr_2018/html/Sandler_MobileNetV2_Inverted_Residuals_CVPR_2018_paper.html
- [4] S. Han, H. Mao, and W. J. Dally, “Deep compression: Compressing deep neural networks with pruning, trained quantization and Huffman coding,” in *International Conference on Learning Representations (ICLR)*, 2016. [Online]. Available: <https://arxiv.org/abs/1510.00149>
- [5] D. P. Hughes and M. Salathé, “An open access repository of images on plant health to enable the development of mobile disease diagnostics,” *arXiv preprint arXiv:1511.08060*, Nov. 2015. [Online]. Available: <https://arxiv.org/abs/1511.08060>
- [6] N. Sankalana, “Cassava leaf disease classification,” Kaggle dataset, 2021, <https://www.kaggle.com/datasets/nimalsankalana/cassava-leaf-disease-classification>.
- [7] S. P. Mohanty, D. P. Hughes, and M. Salathé, “Using deep learning for image-based plant disease detection,” *Frontiers in Plant Science*, vol. 7, p. 1419, 2016.
- [8] B. Jacob, S. Kligys, B. Chen, M. Zhu, M. Tang, A. Howard, H. Adam, and D. Kalenichenko, “Quantization and training of neural networks for efficient integer-arithmetic-only inference,” in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*. IEEE, 2018, pp. 2704–2713.
- [9] R. Krishnamoorthi, “Quantizing deep convolutional networks for efficient inference: A whitepaper,” *arXiv preprint arXiv:1806.08342*, Jun. 2018. [Online]. Available: <https://arxiv.org/abs/1806.08342>
- [10] M. Nagel, M. van Baalen, T. Blankevoort, and M. Welling, “Data-free quantization through weight equalization and bias correction,” in *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*. IEEE, 2019, pp. 1325–1334.
- [11] M. Nagel, M. Fournarakis, R. A. Amjad, Y. Bondarenko, M. van Baalen, and T. Blankevoort, “A white paper on neural network quantization,” *arXiv preprint arXiv:2106.08295*, Jun. 2021. [Online]. Available: <https://arxiv.org/abs/2106.08295>
- [12] G. Hinton, O. Vinyals, and J. Dean, “Distilling the knowledge in a neural network,” *arXiv preprint arXiv:1503.02531*, Mar. 2015. [Online]. Available: <https://arxiv.org/abs/1503.02531>
- [13] A. Romero, N. Ballas, S. E. Kahou, A. Chassang, C. Gatta, and Y. Bengio, “FitNets: Hints for thin deep nets,” in *International Conference on Learning Representations (ICLR)*, 2015. [Online]. Available: <https://arxiv.org/abs/1412.6550>
- [14] S. Zagoruyko and N. Komodakis, “Paying more attention to attention: Improving the performance of convolutional neural networks via attention transfer,” in *International Conference on Learning Representations (ICLR)*, 2017. [Online]. Available: <https://arxiv.org/abs/1612.03928>
- [15] J. Gou, B. Yu, S. J. Maybank, and D. Tao, “Knowledge distillation: A survey,” *International Journal of Computer Vision*, vol. 129, no. 6, pp. 1789–1819, 2021.
- [16] J. H. Cho and B. Hariharan, “On the efficacy of knowledge distillation,” in *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*. IEEE, 2019, pp. 4793–4801.
- [17] S. I. Mirzadeh, M. Farajtabar, A. Li, N. Levine, A. Matsukawa, and H. Ghahmazar, “Improved knowledge distillation via teacher assistant,” in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 34, no. 04, 2020, pp. 5191–5198.
- [18] L. Beyer, X. Zhai, A. Royer, L. Marceeva, R. Anil, and A. Kolesnikov, “Knowledge distillation: A good teacher is patient and consistent,” in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. IEEE, 2022, pp. 10915–10924.
- [19] H. Touvron, M. Cord, M. Douze, F. Massa, A. Sablayrolles, and H. Jégou, “Training data-efficient image transformers and distillation through attention,” in *Proceedings of the 38th International Conference on Machine Learning (ICML)*. PMLR, 2021, pp. 10347–10357. [Online]. Available: <https://proceedings.mlr.press/v139/touvron21a.html>
- [20] K. He, X. Zhang, S. Ren, and J. Sun, “Deep residual learning for image recognition,” in *Proceedings of the IEEE Conference on Computer Vision*

and Pattern Recognition (CVPR). Las Vegas, NV, USA: IEEE, Jun. 2016, pp. 770–778.

- [21] M. Tan and Q. V. Le, “EfficientNet: Rethinking model scaling for convolutional neural networks,” in *Proceedings of the 36th International Conference on Machine Learning (ICML)*. Long Beach, CA, USA: PMLR, Jun. 2019, pp. 6105–6114. [Online]. Available: <https://proceedings.mlr.press/v97/tan19a.html>
- [22] A. Dosovitskiy, L. Beyer, A. Kolesnikov, D. Weissenborn, X. Zhai, T. Unterthiner, M. Dehghani, M. Minderer, G. Heigold, S. Gelly, J. Uszkoreit, and N. Houlsby, “An image is worth 16x16 words: Transformers for image recognition at scale,” in *International Conference on Learning Representations (ICLR)*, 2021. [Online]. Available: <https://openreview.net/forum?id=YicbFdNTTy>
- [23] A. Howard, M. Sandler, G. Chu, L.-C. Chen, B. Chen, M. Tan, W. Wang, Y. Zhu, R. Pang, V. Vasudevan, Q. V. Le, and H. Adam, “Searching for MobileNetV3,” in *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*. Seoul, Korea: IEEE, Oct. 2019, pp. 1314–1324.
- [24] E. C. Too, Y. Li, S. Njuki, and Y. Liu, “A comparative study of fine-tuning deep learning models for plant disease identification,” *Computers and Electronics in Agriculture*, vol. 161, pp. 272–279, 2019.
- [25] E. C. Too, Y. Li, P. Kwao, S. Njuki, M. E. Mosomi, and J. Kibet, “Deep pruned nets for efficient image-based plants disease classification,” *Journal of Intelligent & Fuzzy Systems*, 2019.
- [26] M. Liu, H. Liang, and M. Hou, “Research on cassava disease classification using the multi-scale fusion model based on EfficientNet and attention mechanism,” *Frontiers in Plant Science*, vol. 13, p. 1088531, 2022.
- [27] C. Che, N. Xue, Z. Li, Y. Zhao, and X. Huang, “Automatic cassava disease recognition using object segmentation and progressive learning,” *PeerJ Computer Science*, vol. 11, p. e2721, 2025.